

Towards Security Commit Message Standardization

Sofia Reis
INESC-ID & IST/Técnico
University of Lisbon
Lisbon, Portugal
sofia.o.reis@tecnico.ulisboa.pt

Rui Abreu
INESC-ID & FEUP
University of Porto
Porto, Portugal
rui@computer.org

Corina Păsăreanu
Carnegie Mellon University
USA
pcorina@cmu.edu

Abstract—Timely patch management is crucial for protection against malicious attacks. The lack of quality documentation on security patches hinders triage systems in identifying and prioritizing fixes. Previous work has shown that vulnerability patch detection techniques cannot rely purely on commit messages to detect software patches due to the often inaccurate, incomplete, and missing information in the commit messages. Providing well-structured and quality security patch documentation can improve security awareness, facilitate quicker stakeholder responses to potential vulnerabilities, and allow researchers to produce larger research datasets for security. We investigated the completeness and standardization of 11,036 security commit messages by analyzing the types of information included in those messages and whether or not common practices were applied to create commit messages. We concluded that security commit messages miss important information and do not follow standardization. We propose a standardization solution called SECOM and show the potential of large language models (LLMs) to help us generate more complete and standardized commit messages automatically.

Index Terms—Security Patches, Commit Messages, Software Documentation, Patch Management, Vulnerability Disclosure

I. INTRODUCTION

Delays in deploying patches for known software security vulnerabilities have led to significant cybersecurity attacks [1]–[5]. Timely patch management—ensuring the rapid distribution and deployment of security fixes to users [6]–[9]—is widely recognized as one of the most effective techniques against cyberattacks [6], [10]. However, a critical challenge persists: the *lack of efficient patch triage systems* to identify and prioritise security patches [3], [7], [11], [12]. Current processes remain primarily manual, time-consuming, and susceptible to overlooking essential security patches. An ideal patch management process should incorporate an effective audit system to identify and assess patches [3].

Previous work has investigated using commit messages and code changes for automated patch detection [12]–[15]. Studies have concluded that software vulnerability management techniques should not rely solely on commit messages or patch documentation due to their frequent inaccuracies and incompleteness [15], [16]. E.g., 62% of commit messages used to “silently” patch software vulnerabilities did not include any security-related words [14]. Silent fixes occur when vendors patch vulnerabilities without explicitly acknowledging them in release notes, commit messages or other types of patch documentation [14]. While this practice aims to protect vendors

and systems, it also reduces the general knowledge pool of those who understand the vulnerability, limiting defenders’ awareness and leaving users unprotected. According to the CERT Coordinated Vulnerability Disclosure (CVD) guide, silent patches should be avoided since knowing the existence of vulnerabilities and their patches is often the key driver to effective patch deployment [17]. Moreover, inadequate patch documentation hinders triage systems’ ability to effectively detect and prioritise security fixes [18].

Motivation. Security commit messages associated with vulnerability patches are often cryptic and hard to understand [14]. Table I shows examples of security commit messages (col. Message) used to document patches to known software vulnerabilities (col. VulnID) and the patch commit key (col. SHA). Message 1 (“.”) is an example of a cryptic message as it does not provide any information. From messages, it is possible to infer that a defect in the code is being fixed, but nothing more than that. Message 3 is an internet meme called “Rickrolling”¹, usually used to prank people with the “Never Gonna Give You Up” song. Although humorous, it once again lacks any relevant information regarding the vulnerability and respective patch. Most of the cases presented would not be detected by triage systems, leaving users unaware of the new patches and exposed to fixed vulnerabilities. Message 4 provides a brief description of the patch and some relevant information about it (e.g., it fixes a “security issue” related to “passwords”). The patch fixes an “Improper Authentication” weakness (CWE-287) with a severity (CVSS) score of 8.4 in 10—which is not explicit in the message.

Study. While some advocate for minimalist security commit messages and patch notes, others emphasize the importance of including detailed information to support effective triage systems [11], [12], [17] and accelerate patch management [11], [17]. In this work, we investigate: (1) what types of information are included or missing in security commit messages (completeness); (2) whether security engineers follow best practices when writing these messages (standardization); and (3) the potential of Large Language Models (LLMs) to generate more complete and structured commit messages. To this end, we conduct an empirical analysis of 11,036 security commit messages.

¹“Rickrolling” meme details at <https://knowyourmeme.com/memes/rickroll>

TABLE I: Examples of Real Security Commit Messages

#	Commit Message	Vulnerability ID	SHA
1	.	CVE-2019-13568	ac80033
2	Minor patch.	OSV-2020-2108	a8bf10e
3	Code refactoring.	GHSA-4fc4-4p5g-6w89	d158413
4	<scratchsig><script>location='https://www.youtube.com/watch?v=dQw4w9WgXcQ';</script></scratchsig>	CVE-2020-15179	4160a39
5	Fixed security issues with passwords entered via a prompt	CVE-2022-35928	6876185

Note: The commit associated with CVE-2022-35928 (6876185) is no longer publicly accessible on GitHub at the time of writing. However, its commit message was archived in our dataset and is included here for illustrative purposes.

Results. Despite the expectation that security-related commit messages clearly document corrective actions, our analysis reveals that only 58.1% explicitly reference such actions—highlighting a widespread lack of clarity regarding the changes being made. Mentions of flaws, vulnerability identifiers, and links to issue trackers are also rare, limiting the contextual richness needed for effective patch triage and prioritization. While some engineers follow recommended practices, our findings show that adherence is inconsistent and that security commit messages frequently lack standardization. To address this, we introduce **SECOM**, a structured convention for writing security commit messages that encapsulates essential metadata and improves message quality and consistency. Furthermore, we explore the potential of LLMs to automatically generate more complete and standardized messages. Our results show that LLMs can significantly enhance message informativeness and structure, especially when combined with curated metadata. Together, SECOM and LLM-assisted generation aim to support better documentation practices, facilitate vulnerability management, and strengthen the foundations of secure software development.

Contributions. This work makes three main key contributions: (1) an empirical analysis of over 11k security commit messages associated with publicly disclosed vulnerabilities, in which we analyze the information they contain and evaluate their alignment with established best practices; (2) the proposal of **SECOM**, the first convention specifically designed to standardize and enrich security commit messages; (3) an evaluation of LLM-based commit message generation, demonstrating its potential to enhance message completeness and structure at scale. We also release datasets to support future research in security patch management and commit message generation. SECOM is publicly documented to facilitate adoption. To learn how to apply and configure it within your development workflow, visit <https://security-commits.org/>. SECOM was validated through discussions with two Open Source Security Foundation (OpenSSF) working groups and refined based on feedback from the broader open source security community.

Replication Package. All the scripts to collect, manipulate, and preprocess the data; the dataset of security commit messages leveraged in the study; and, notebooks to report the analysis discussed in this paper are available at <https://figshare.com/s/c6f5b36711ed03816674>.

II. RESEARCH QUESTIONS

This study addresses three research questions (RQ1–RQ3) through an empirical analysis of security commit messages linked to publicly disclosed vulnerabilities.

RQ1. Which types of information can be extracted from security-related commit messages? Existing research indicates that commit messages contain valuable information for security detection [12]–[14], [16], [18], [19], despite data quality concerns [15]. However, the specific types of information within security-related commits remain largely unexplored. This study leverages Natural Language Processing (NLP) techniques to analyze, identify, and categorize the information embedded in security commit messages.

RQ2. How do security engineers apply best practices when writing security commit messages? While guidelines exist for writing general commit messages [20]–[23], there are no established standards tailored for security-related commits. Prior work shows that 44% of general commit messages [20] and 56.7% of security commit messages [18] lack clarity and require improvement. This study investigates how security engineers apply existing guidelines [21], [22], [24] and examines how these practices could be adapted to enhance the structure and completeness of security commit messages.

RQ3. How do LLMs influence the completeness of security commit messages? Since adhering to common practices and standards can be time-consuming, we evaluate the potential of LLMs to enhance the completeness of security commit messages by leveraging their ability to follow instructions and summarize code effectively [25]–[28].

III. SECURITY COMMIT MESSAGES ANALYSIS

To answer RQ1 and RQ2, we present the methodology and findings of an exploratory empirical study on security commit messages from patches addressing known vulnerabilities.

A. Dataset Collection and Preprocessing

This section outlines the steps taken to create a dataset of security commit messages. All tools used for data collection and preprocessing are available in our replication package. The dataset includes data released up **until August 12, 2022**. Security commit messages for the exploratory analysis were collected following the procedures described below.

1) *Vulnerability Metadata Collection*: Public vulnerability databases, such as the National Vulnerability Database (NVD) [29] and the Open-Source Vulnerability (OSV) database [30], aggregate reports for thousands of known vulnerabilities. Our OSV database dump contains 30,091 vulnerability reports for open-source vulnerabilities across various ecosystems: 28.6% from GitHub Advisories, 25.7% from Linux, 11.1% from PyPI, 8.5% from NPM, 7.8% from OSS-Fuzz, and the remaining 18.3% from other sources (e.g., Maven, RubyGems, Go, and more). OSV includes only reports for vulnerabilities published after 2005 and focuses on a limited set of ecosystems. In contrast, the NVD includes vulnerability reports from as far back as 1999, with no restrictions on the ecosystems covered. As a result, we also incorporated the NVD database in our study, collecting a total of 181,614 vulnerability reports. In total, we gathered 211,705 vulnerability reports from both databases.

2) *Collection and Preprocessing of Security Patches*: Each vulnerability report from both data sources includes a reference to the associated fix or patch², when available.

Collection. To identify the commits associated with patching security vulnerabilities, we filtered out all vulnerability reports that did not include references to commits. We found that only 10,010 out of 30,091 OSV reports and 9,953 out of 181,614 NVD reports included references to commits, meaning that only approximately 9% of vulnerability reports across these databases have fixes available. In addition, we observed that commit references are typically available through platforms such as GitHub, Bitbucket, SVN, and other similar services. In this study, we focus exclusively on vulnerability reports that include references to security patches available on GitHub, which account for over 80% of the commits extracted from all vulnerability reports. In total, we identified GitHub commit references in 8,670 NVD reports and 9,576 OSV reports.

Preprocessing. To collect the commit messages, we used the GitHub API, which requires the repository’s *owner*, *repository name*, and *version* (or SHA key) associated with the vulnerability. Due to incomplete information, we could not retrieve the necessary data—e.g., when the commit link pointed to the master branch instead of a specific SHA key. It was not possible to confirm whether the master version corresponded to the vulnerability. As a result, we excluded these reports, resulting in a dataset of 8,405 patches for NVD (a 3% reduction) and 9,466 patches for OSV (a 1% reduction).

Merging and Cleaning. After normalization, we merged both data sources into a single dataset of 17,871 patches, preserving the metadata from the vulnerability reports. Since many vulnerabilities appear in both NVD and OSV, we applied the following heuristics to address duplicates:

- (1) **Duplicated Entries**: We removed security patches missing vulnerability score values in one of the sources, accounting for 18% of the data points;

- (2) **OSV Aliases**: OSV includes an “aliases” field that lists IDs of the same vulnerability from other databases. We removed all NVD entries (19% of data points) whose IDs appeared as aliases in OSV. OSV was prioritized, as prior research has highlighted documentation issues in NVD [32]–[34] and OSV’s efforts to address them.

- (3) **Vulnerabilities Fixed by the Same Patch**: We removed entries where a patch addressed multiple vulnerabilities, affecting different codebases but resulting in separate vulnerability reports (13% of data points). The final dataset consists of 10,254 security patches.

3) *Collecting and Preprocessing Commit Messages*: Vulnerabilities can be addressed by a single commit (single-commit patch) or multiple commits (multi-commit patch). Of the 10,254 security patches, 88.6% (9,083) are single-commit and 11.4% (1,170) are multi-commit patches. In total, we extracted 11,809 security commits. Commit messages and metadata were collected using the GitHub API and underwent several preprocessing steps:

- (1) **Unavailable Commits**: A total of 334 commits (corresponding to 160 vulnerability reports) were no longer accessible at the time of metadata collection and were therefore removed from the dataset;

- (2) **Duplicate Messages**: We identified duplicate commit messages from vulnerability reports referencing the same fix across different branches. For example, the GHSA-273r-mgr4-v34f³ advisory referenced a commit for each branch where the vulnerability was fixed. Since the commit messages were identical, we retained only one instance, removing 270 duplicates. This resulted in 11,205 security commit messages;

- (3) **Automated Commit Messages**: Following prior work [20], we searched for non-human-generated message patterns. Examples include messages like `<original commit message>` (cherry-picked from commit `<commit>`) or automated merge messages (e.g., `merge pull request from <vuln-ID>`). We excluded 126 such automated messages, resulting in 11,079 security commit messages;

- (4) **Non-English Messages**: We used the `langdetect` tool⁴ to identify non-English commit messages. The model initially detected 1,311 (11.8%) messages as non-English. Since the tool can produce inaccurate results for short or ambiguous texts, we manually reviewed these messages. After validation, we removed 43 non-English commit messages, such as “导入mysql db时报错” and “用户上传格式限制”. After these preprocessing steps, we retained a final number of 11,036 messages.

Results. We collected a total of 11,036 security commit messages corresponding to 9,943 unique security patches, covering 278 distinct publicly disclosed vulnerabilities.

³GHSA-273r-mgr4-v34f report available at <https://github.com/advisories/GHSA-273r-mgr4-v34f>

⁴`langdetect` infers the language of natural text across 55 languages. Available at <https://github.com/Mimino666/langdetect>.

²CVE-2022-35928 Report (Section “References to Advisories”): <https://nvd.nist.gov/vuln/detail/CVE-2022-35928>

TABLE II: Category names, rationale, and entity examples

Type	Category	Rationale	Entity Examples	Rules
SEC	SECWORD	Security-relevant terms are often used to describe the vulnerability itself and the corresponding patch.	ldap injection, crlf injection, improper validation, command injection, cross-site scripting	1718
	VULNID	Vulnerability identifiers—such as CVE, GHSA, OSV, and PyPI advisories—are commonly used in commit messages to reference security issues across different ecosystems.	GHSA-269q-hmxg-m83q, CVE-2016-2512, OSV-2016-1	16
	CWEID	Vulnerabilities are typically classified according to a specific weakness defined in the Common Weakness Enumeration (CWE) taxonomy.	CWE-119, CWE-20, CWE-79, CWE-189	6
	SEVERITY	Vulnerabilities are typically assigned a severity rating to indicate their potential impact.	low, medium, high, critical	4
	DETECTION	Vulnerabilities can be identified either manually or through the use of specialized tools.	Manual, CodeQL, Coverity, OSS-Fuzz, libfuzzer	8
COM	SHA	Commit hashes referencing earlier versions where the vulnerability was introduced, as defined by the OSV Schema [31].	f8d773084564, 228a782c2dd0	5
	ACTION	A commit typically represents an action—such as fixing a vulnerability in the context of security (i.e., <i>corrective maintenance</i>).	fix, patch, change, add, remove, found, protect, update, optimize, mitigate	16
	FLAW	Fixing a security vulnerability typically involves correcting an underlying flaw in the code.	defect, weakness, flaw, fault, bug, issue	10
	ISSUE	The GitHub issue or pull request number is sometimes included in the commit message and can offer additional context about the vulnerability.	#2, #13245	1
	EMAIL	Reviewer and author contact emails often appear after tags like <code>Reported-by</code> and are useful for identifying points of contact.	johndoe@gmail.com, johndoe@yahoo.com, johndoe@hotmail.com, johndoe@outlook.com ¹	1
	URL	References to reports, blog posts, and bug trackers offer additional information about the vulnerability.	https://www.htbridge.ch/advisory/multiple_vulnerabilities_in_mantisbt.html	1
	VERSION	Software versions are frequently mentioned in commit messages.	3.1.0, v3.2, v2.6.28, 1.6.3, 2.1.395	3
Type SEC: Security specific categories; Type COM: Commit specific categories. ¹ Dummy e-mails generated for compliance with General Data Protection Regulation (GDPR).				

B. Data Extraction (RQ1)

This section details how key information was extracted from security commit messages and summarises the main findings.

Methodology: The following subsections detail RQ1 steps.

1) *Text Analysis:* We used Named Entity Recognition (NER) to extract key information from security commit messages. NER is a widely used NLP technique for identifying and classifying entities in unstructured text based on predefined rules or patterns [35]–[37]. Unlike text classification, which analyzes overall text characteristics (e.g., sentiment analysis), NER provides fine-grained understanding by extracting specific *entities*. An *entity* is a word or phrase referring to a *category* (e.g., “Netflix,” “Google,” and “Apple” under the *Company* category). Designing an effective NER system requires defining domain-specific categories and entity values. NER has been applied across various fields, including biomedical sciences [38], pharmacology [39], and security for extracting product names and versions from vulnerability reports [40].

2) *Category Design:* We defined two data groups: (1) **security-specific (Type SEC)**, covering categories commonly found in security commit messages and vulnerability reports (e.g., SECWORD, VULNID, CWEID); and (2) **commit-specific (Type COM)**, covering categories typical of commit messages (e.g., SHA, ACTION, FLAW, ISSUE). In total, we designed **12 entity categories**: 5 security-related and 7 commit-related. These categories were selected based on domain knowledge, prior work, and manual inspection of real-world patches.

Table II summarizes the categories, providing their rationale, examples, and the number of implemented pattern-based rules.

3) *NER Pipeline:* To extract entities, we used the Python library `spacy`⁵, which provides end-to-end pipelines for various NLP tasks, including NER. We developed a custom pipeline tailored for the security domain (see Figure 1). **Tokenization:** Each security commit message is tokenized—i.e., split into meaningful units called *tokens*—using `spacy`’s built-in tokenizer.⁶ **Part-of-Speech Tagging:** Tokens are annotated with grammatical categories (verbs, nouns, adjectives, etc.) using `spacy`’s POS tagger⁷, based on a pre-trained English model, `en_core_web_lg`⁸. After obtaining tokens and POS tags, the pipeline applies custom rules designed for security commit messages. Rule-based entity recognition is implemented using `spacy`’s *Entity Ruler*⁹, enabling flexible and efficient extraction of security-related entities. The entire pipeline processes 11,036 security commit messages in approximately 1.56 seconds on an Apple M3 machine.

4) *Rules:* Fixing security vulnerabilities involves an ACTION—one of the entity categories defined in Table II. These actions are often expressed with verbs such as “fix,” “update,” “patch,” and “mitigate.” To capture this variability,

⁵`spacy` Natural Language Processing library, available at <https://spacy.io/>

⁶Tokenizer documentation: <https://spacy.io/api/tokenizer>. Additional details: <https://spacy.io/usage/linguistic-features#tokenization>.

⁷Part-of-Speech (POS) tagger documentation: <https://spacy.io/api/tagger>

⁸Model: https://spacy.io/models/en#en_core_web_lg

⁹Entity Ruler: <https://spacy.io/usage/rule-based-matching#entityruler>

Fig. 1: Extraction Pipeline

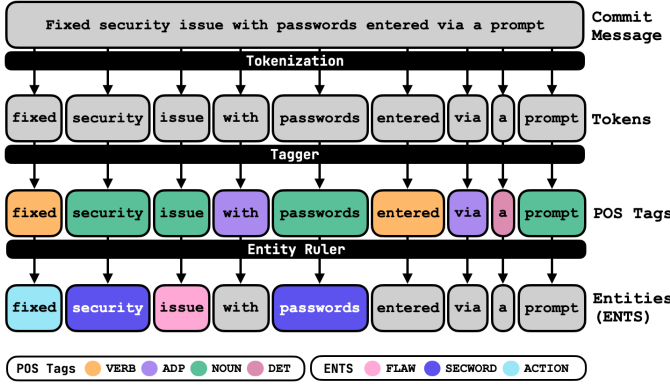


TABLE III: NER rule examples

ID	Label	Rule
R1	ACTION	{ "label": "ACTION", "pattern": [{ "LOWER": { "REGEX": "fix.*" }, "POS": "VERB" }] }
R2	VULNID	{ "label": "VULNID", "pattern": [{ "LOWER": "osv" }, { "IS_PUNCT": true }, { "SHAPE": "dddd" }, { "IS_PUNCT": true }, { "LIKE_NUM": true }], "id": "CVE" }

we designed rules to identify verbal forms. Table III shows two examples from our entity ruler. The first rule targets all inflected forms of “fix” (e.g., “fix,” “fixing,” “fixed,” “fixes”) by matching tokens labeled as VERB during POS tagging. The second rule addresses vulnerability identifiers. As the open-source security ecosystem grows, platforms like PyPI, NPM, and RubyGems have introduced new ID formats beyond the traditional CVE pattern (e.g., CVE- $\backslash\{4\}-\backslash\{4,7\}$). To handle this diversity, we implemented pattern-matching rules under the VULNID category. For example, Rule R2 (Table III) captures OSV-style identifiers (e.g., OSV-2023-27¹⁰) used by tools such as Google’s OSS-Fuzzer. We developed **1,718 extraction rules** targeting security-related terms (SECWORD), inspired by prior work [41], [42]. These rules were iteratively refined through manual inspection and validation of real-world security commit messages. During this process, we identified two main challenges: (1) erroneous extractions, such as broken tokenization of GitHub Advisory IDs, which required refining existing rules; and (2) missing extractions, which led us to expand the entity category list and develop new rules. This analysis also revealed recurring anti-patterns specific to security commit messages, which will be discussed later. The full set of rules is available in our replication package (labeller/entity_ruler/patterns.jsonl).

Findings. This section presents the findings for RQ1, based on the results of our extraction pipeline (Figure 1) applied to 11,036 security commit messages. Table IV details the extracted entities, reporting the total number of entities per

TABLE IV: Extraction Results (RQ1)

Extraction Results for security-specific (SEC)			
Category	#Entities	#Commits	% Commits
SECWORD	16167	6757	61.2%
VULNID	1799	1330	12.1%
DETECTION	629	374	3.4%
SEVERITY	142	118	1.1%
CWEID	27	25	0.2%
Total	18766	7316	66.3%
Extraction Results for commit-specific (COM)			
Category	#Entities	#Commits	% Commits
ACTION	12340	7188	65.1%
EMAIL	4738	2087	18.9%
SHA	4791	1473	13.4%
FLAW	4236	2761	25.0%
ISSUE	3561	2805	25.4%
URL	1182	930	8.4%
VERSION	658	571	5.2%
Total	31505	9240	83.7%
Extraction Results for security- and commit-specific (SEC & COM)			
Category	#Entities	#Commits	% Commits
Total	50271	10312	94.4%

category (#Entities), the number of unique commits containing each entity type (#Commits), and the corresponding percentage (%Commits). Results are organized into two groups: **security-specific (SEC)** and **commit-specific (COM)**.

- **Security-specific data (SEC).** The pipeline extracted entities across all five security-specific categories, yielding **18,766** entities from **7,316** unique commits, covering **66.3%** of the dataset. The most prevalent category was SECWORD, with **16,167** entities extracted from **6,757** commits (61.2%), capturing terms related to vulnerabilities and fixes (e.g., “injection,” “xss,” “overflow”). Although each commit addressed a known vulnerability, only **1,330** messages included explicit VULNID references (e.g., CVE or GHSA IDs), representing 12.1% of the dataset. These identifiers are essential for linking fixes to official vulnerability reports, and their absence limits traceability and complicates triage. Similarly, only **25** commit messages contained CWEIDs, suggesting weakness classifications are rarely included despite their importance for understanding vulnerability types. In some cases, informal terms (e.g., “xss”) may allow indirect inference of CWE types (e.g., CWE-79). Severity levels were found in just **118** messages (1.1%), and detection method mentions (DETECTION) appeared in only **374** commits (3.4%). These results show that key metadata—including severity, detection method, and formal classifications—are often omitted, limiting the utility of commit messages for security analysts, triage systems, and automated tools.

¹⁰<https://osv.dev/vulnerability/OSV-2023-27>

- **Commit-specific data (COM).** The extraction pipeline identified entities across all seven commit-specific categories, totalling **31,505** entities from **9,240** unique commits, covering **83.7%** of the dataset. Commit messages in security contexts are typically expected to reflect an *action* or *change* related to fixing a vulnerability (i.e., *corrective maintenance*). The most frequently identified category was ACTION, with **12,340** entities extracted from **7,188** commits (65.1%). However, over one-third of messages lacked a clear corrective action, suggesting limited clarity in describing changes. The pipeline also captured **4,738** EMAIL entities across **2,087** commits (18.9%), identifying contributors or reviewers. Mentions of FLAW terms (e.g., "bug," "issue," "defect") appeared in **2,761** commits (25.0%), indicating that many messages omit explicit references to the underlying problem. Additional markers such as ISSUE references and URLs were relatively rare: **2,805** commits (25.4%) linked to an issue or pull request, and only **930** (8.4%) included external documentation. The low frequency of these elements highlights a broader gap in the completeness and traceability of security commit documentation.

No Entities Extracted. Our extraction tool identified 50,271 entities across 10,312 of the 11,036 security commit messages, covering 94.4% of the dataset. To understand the remaining 5.6% of messages without extracted entities, we conducted a qualitative review. This analysis revealed 3 recurring issues:

(1) **Poorly Documented.** Many commit messages were vague, overly generic, or lacked sufficient detail to convey the purpose of the change, offering limited value for security analysis. Examples include single words or ambiguous phrases such as "update", "...", ":arrow_up:", "Refactor", "minor fixes", and "minor patch", all failing to communicate the nature or motivation of the change.

(2) **Non-Security Related.** Several commit messages, while technically detailed, lacked an explicit connection to security. For example, the message linked to OSV-2021-1606 states: *"Support progressive event for dc. This implements the progressive API event for the dc image. This is currently only supported for vardct without extra channels; for modular and extra channels, it's only supported if squeeze is used, and it may not correctly work with flush yet in that case."* While informative regarding functionality, it offers no indication of addressing a security concern. Another example is *"core: parser – additional check for content length spanning next line"* which may suggest a safeguard against malformed input. However, the absence of security-specific language makes its intent ambiguous. These cases highlight that even technically descriptive commit messages can be hard to classify as security-relevant without more direct indicators.

(3) **Misspelling Issues.** Some commit messages contain spelling errors that can hinder the recognition of security-related terms. For instance, "sanitizing user input" includes a typo that may reduce the effectiveness of NER models relying on lexical cues. Addressing these issues—through preprocessing techniques like spell correction or by training

models to handle common misspellings—could enhance the performance of future NER systems for security.

Conclusions. These results provide insights into the current structure of security commit messages, highlighting both the types of information included and the critical gaps that persist. Key security metadata—such as vulnerability identifiers, weakness classifications, severity levels, and detection methods—is often absent or inconsistently documented. These omissions limit the utility of commit messages for automated vulnerability detection, patch triage, and security analysis. Encouraging more structured and complete documentation could significantly improve traceability, enhance security tooling, and support timely, accurate patch management.

C. Common Practices Analysis (RQ2)

Following best practices is a recognized strategy for producing clear, effective commit messages [20]–[23]. This study examines whether such guidelines are applied to security patches and how they might improve message structure and clarity. Based on five widely cited sources [20]–[24], we identified seven core recommendations and developed corresponding compliance checks. Our analysis focuses on **structural aspects**—such as the presence of a header, body, issue references, and contributor metadata—rather than **semantic content** (e.g., explaining what, why, and how, as emphasized in [20]). Table V summarizes these checks and their sources. Additional checks are available in our replication package.

Findings. The following summarizes key gaps and opportunities for improving the structure of security-related commits.

C1. Prefix Usage (Conventional Commits): Prefixes in commit headers—such as "fix" or "patch"—recommended by the Conventional Commit Convention [21] facilitate tracking and categorization. However, only 4.10% of security commit messages follow this convention, revealing a gap in standardization within the security domain.

C2. Header Usage: Although 100% of commit messages include a one-line header, only 43.53% contain both security-relevant terms (SECWORD) and action-oriented language (ACTION). This suggests that while formatting

TABLE V: Commit Message Best Practices

ID	Common Practice	Standard
C1	The header should be prefixed with a type.	[21]
C2	The message should have a one-line header/subject.	[21], [22], [24]
C3	The message should have a body.	[22], [24]
C4	The message should mention the author's contact (signed-off-by and authored-by).	[22], [24]
C5	The message should mention the reviewer's contact (reviewed-by).	[22], [24]
C6	The message should mention the reporter's contact (reported-by).	[22]
C7	The message should mention references to issues or pull requests.	[24]
C8	The message should include references to bug trackers.	[24]

is respected, many headers lack the clarity needed to indicate both the security relevance and the nature of the change. Greater consistency would support faster identification of security patches in developer workflows and automated tools such as `git reflow`.

- C3. Body Usage:** Only 52.56% of commit messages include a body, meaning 5,235 (47.44%) omit it entirely. Among those with a body, just 36.61% contain security-related terms. This represents a missed opportunity to document the vulnerability, its impact, and the applied fix in a structured manner. Encouraging consistent use of the body would improve traceability, support future audits, and enhance patch maintainability.
- C4. Sign-off Practices:** Sign-off tags establish accountability and authorship. Only 17.52% of security commit messages included a `Signed-off-by` tag, and just 1.86% used `Authored-by`. Although `Signed-off-by` is the most common, its low adoption suggests that explicit responsibility acknowledgement is not standard practice. The minimal use of `Authored-by` indicates that collaborative contributions are rarely documented. Broader adoption could enhance transparency and ownership clarity, especially in collaborative and open-source projects.
- C5. Reviewer Information:** Only 3.33% of security commit messages included a `Reviewed-by` tag, indicating that reviewer information is rarely recorded. This lack of attribution makes it harder to understand the rationale behind a fix and to trace who validated the change. Including reviewer details would support follow-up discussions and help explain decisions, particularly when patches are revisited or vulnerabilities reappear.
- C6. Reporter Information:** Only 6.52% of security commit messages included a `Reported-by` tag, making it difficult to credit the person who discovered the issue or follow up with them for clarification. Including this information helps acknowledge the contributor and trace the origin of the fix, offering insight into why the issue was deemed a vulnerability.
- C7-8. Issue and Bug Tracker References:** References to issues (25.42%) and bug trackers (1.78%) provide valuable context for security patches but remain limited. Including links to discussions, severity assessments, or bug reports would strengthen traceability and help developers understand the rationale behind security fixes.

Conclusions. While some best practices are observed, security commit messages frequently omit critical structural and contextual elements. The low adoption of best practices reveals a lack of consistency in how security-related changes are documented. This inconsistency hinders traceability, complicates audits, and reduces the effectiveness of downstream tools. Improving adherence to these practices would enhance the clarity and maintainability of security fixes. As software supply chains grow in complexity and security demands increase, clear, complete, and standardized commit messages will be essential for effective vulnerability management.

IV. STANDARDIZATION AND LLMs

Findings from the first part of our study highlight the need for greater standardization in security commit messages. To address this, we introduce **SECOM**, a standard for security commit messages. SECOM was initially developed based on insights from RQ1 and RQ2, then shared with the open-source community. Feedback from practitioners was used to refine and improve the standard iteratively. This section presents an overview of SECOM, detailing its core principles and recommended practices. We also describe the methodology used to address RQ3, which evaluates the ability of LLMs to generate key components of security commit messages.

A. SECOM: Convention for Security Commit Messages

SECOM is the first comprehensive convention for writing clear, structured, and security-focused commit messages. It has been adopted as a standard practice in the “OpenSSF Compliant Automated Vulnerability Fix Campaign” and was used to generate thousands of commits addressing “CVE-2021-26291”¹¹. The standard was designed based on (1) widely recognized guidelines for writing effective commit messages [21]–[24]; (2) our empirical analysis of real-world security commit messages from known security patches (RQ1 and RQ2); (3) feedback from members of the open-source security community (reported in this section).

Listing 1: SECOM Convention

```
vuln-fix: <header / subject> (<Vuln-ID>)

<body>
# What:    Describe the vulnerability.
# Why:     Explain its impact or risk.
# How:     Describe how it was fixed.
</body>

[For each identified weakness:]
Weakness:  <Weakness Name or CWE-ID>
Severity:  <Low | Medium | High | Critical>
CVSS:      <CVSS Score (0-10)>
Detection: <Detection Method>
Report:    <Link to Advisory or Report>
Introduced-in: <Commit Hash of Introduced Vuln>
[End Weakness Block]

Reported-by:    <Name> (<Contact>)
Reviewed-by:    <Name> (<Contact>)
Co-authored-by: <Name> (<Contact>)
Signed-off-by:  <Name> (<Contact>)

Bug-tracker:    <Bug Tracker URL>
Resolves:       <Issue or PR Number>
See also:       <Related Issue or PR Number>
```

Convention Design. Table VI outlines the SECOM fields, their descriptions, and rationale. These fields promote clarity, consistency, and traceability in security commit documentation. SECOM (Listing 1) is organized into **five main** sections:

¹¹<https://github.com/JLLeitschuh/security-research/issues/8>

TABLE VI: Fields description, Best Practice(s) and Rationale.

Field	Description	Best Practice(s)	Rationale
Type	Signals that the commit addresses a vulnerability.	Use <code>vuln-fix</code> ” at the beginning of the header to indicate a security-related fix. Optionally, include a scope (e.g., <code>vuln-fix(auth)</code> ”) to specify the affected component.	Each commit should be assigned a type [21] to facilitate the identification and classification of vulnerability fixes. Adherence to the Conventional Commits format (<code><type>(<scope>)</code> ”) was observed in only 4.10% of analyzed commits. This limited adoption suggests an opportunity for future standardization efforts to improve the traceability of security patches. The term <code>vuln-fix</code> was adopted following a recommendation from the Google OSV team.
Header	Summarizes the commit purpose in one single-line [22], [24].	It should be written in imperative form, capitalized, avoid a trailing period, stay concise (ideally around 50 characters, maximum 72) and include action-oriented and security-relevant terms.	While 100% of security commit messages included a header, only 43.53% incorporated both a security-related term and an action-oriented verb. This suggests that a substantial proportion of headers may not provide sufficient information to support the effective identification of security-related changes.
Vuln-ID	Designed to standardize the tracking and referencing of known security issues.	Include the vulnerability identifier when available (e.g., CVE, OSV, GHSA, or other relevant formats).	Only 12.1% of commit messages included a reference to a vulnerability identifier. Additionally, 7 out of 10 participants indicated that including the <code>vuln-ID</code> in the commit message is important. Incorporating it into the header can facilitate the localization of the commit responsible for patching the vulnerability when using <code>reflog</code> or <code>shortlog</code> .
Body	Describes the vulnerability (what), its impact (why), and how the patch addresses it (how) [20].	Aim for a total of around 75 words in the commit body, with approximately 25 words dedicated to each key aspect: what, why, and how. Be sure to include security-relevant terms in the description to enhance clarity and traceability.	Our empirical analysis showed that only 52.56% of commit messages included a body, and just 36.63% incorporated security-related terms. Incorporating a clear and complete body will enable the ability of both humans and automated tools to assess the nature and intent of the commit.
Weakness	Provides a standardized classification of software weaknesses.	Include the Common Weakness Enumeration (CWE) identifier or name when applicable.	The CWE taxonomy is widely used to categorize and communicate vulnerability types. Our analysis found that only 0.2% of commit messages included a reference to a CWE identifier. Including this information enables software patch management teams to better prioritize and handle patches based on the weakness nature.
Severity	Categorizes the impact of the vulnerability.	Include the vulnerability severity (Low, Medium, High, Critical).	Commonly used for patch prioritization. Only 1.1% of commit messages mention severity levels. Severity information can motivate security teams to perform patch management faster in the case of critical vulnerabilities [17]. Without explicit severity, downstream users must independently assess urgency, delaying response times.
CVSS	Provides a standardized numerical score (0–10) that quantifies the severity of the vulnerability based on the Common Vulnerability Scoring System (CVSS).	Include the CVSS base score to indicate the technical impact of the vulnerability.	CVSS scores are widely adopted in the security community for assessing vulnerability severity. Including a CVSS score can help developers and security teams prioritize fixes more effectively [17].
Detection	Indicates how the vulnerability was discovered (e.g., static analysis tool, manual review).	Include the detection method or tool used (e.g., CodeQL, Veracode, Manual).	7 out of 10 participants found this field valuable. Our analysis found that only 374 commit messages referenced a detection method. Including it can support reproducibility and help validate detection tools.
Report	Provides a link to an external vulnerability report, advisory, or blog post that offers additional context.	Include a URL to supporting material, such as a security advisory or proof-of-concept.	Half of the participants (5 out of 10) expressed interest in this field. External reports can help bridge information gaps in the commit message by detailing the exploit scenario, impact, or remediation steps.
Introduced in	Identifies the commit where the vulnerability was originally introduced.	Include the SHA of the introducing commit when known.	Endorsed by the Google OSV team as a key element of the OSV Schema [31]. Our analysis found SHA references in 1,473 commit messages, indicating that developers sometimes include commit hashes for traceability. While not always pointing to the introducing commit, this practice suggests potential for improving root-cause analysis when consistently applied.
Reported-by	Name and contact information of the person who reported the vulnerability.	Include the name and email of the person who reported the vulnerability.	Only 6.52% of messages include the reporter’s contact, which is crucial for validation and follow-up during the vulnerability review.
Reviewed-by	Name and contact information of the person who reviewed the patch.	Include the name and email of the person who reviewed the vulnerability and patch.	Only 3.33% of messages have the reviewers’ contact. Including it would support accountability and the long-term assessment of patch effectiveness, when issues re-emerge and patches need to be revisited.
Signed-off by/Author-ed-by	Name and contact information of the person who fixed the vulnerability.	Include the name and email of the person who fixed the issue. For multiple contributors, use the “Co-Authored-by” tag.	Only 17.52% of commit messages were signed off by the respective authors. The absence of sign-offs can also make it harder to establish clear ownership and can complicate auditing and compliance efforts.
Bug-tracker/Issues	Links to any sources of additional context.	Include links to relevant bug tracker issues, pull requests, or discussion threads that provide additional context.	Important to document and discuss the problem, its impact and know which people were involved. In our empirical analysis, we extracted URLs from a total of 930 commits.

(1) **header**, prefixed with the type `vuln-fix`, contains a concise summary of the change along with a vulnerability identifier, when available. This format facilitates easy identification and filtering through tools such as `git reflog`; (2) **body**, which describes the vulnerability (**what**), its impact (**why**), and the applied fix (**how**); (3) **metadata**, which includes details such as the type of weakness (CWE-ID), severity level, CVSS score, detection method, report link, and the version or commit in which the vulnerability was introduced; (4) **contacts**, listing the names and email addresses of the reporter(s), reviewer(s), author and co-author(s); and, (5) **references** to relevant issue trackers or bug reports. Each section should be separated by a newline for readability and consistency. SECOM also follows the principle that security patches should be atomic [23]; while a single fix may address multiple weaknesses, patches requiring distinct changes should be committed separately.

Open-Source Security Community Feedback: This section describes how we informally collected feedback from the open-source security community, including members of the Google OSV team, OpenSSF contributors, and researchers in this area. OpenSSF is an open and collaborative community of security engineers from across the industry—including organizations such as Google, the Linux Foundation, Intel, and Red Hat—united by the mission to improve the security of open-source software [43]. SECOM was presented and discussed in detail within two OpenSSF working groups—*Best Practices* and *Vulnerability Disclosure*. At the conclusion of our presentation, we invited security engineers to share their feedback on the convention via a form—without collecting any personal data—and through an open discussion. The form included two questions: **Q1** *What do you think about the proposed set of fields (e.g., Vuln-ID, Severity, etc.)?* and **Q2** *Would you use SECOM as a standard practice in your own work or advocate for its adoption within your team?*. Additionally, we provided an open-text field for participants to offer suggestions or general feedback. The feedback and its implications were incorporated into the final design of SECOM, as reflected in Table VI. For example, we defined “vuln-fix” as the type to be used in security commit messages based on feedback provided by the Google OSV Team. Feedback from the security community indicates strong support for SECOM, with 7 out of 10 participants expressing a positive outlook on its adoption, while the remaining 3 were undecided. Among the proposed fields, the vulnerability ID, CWE-ID, and severity were consistently identified as the most valuable to include in commit messages. In contrast, CVSS scores and report links were seen as less critical. Participants who were unsure about SECOM’s adoption cited concerns about practical implementation. One participant noted, “It’s going to be difficult to get adherence to a standard for exceptional cases, rather than the common case. Folks won’t be practicing this style daily, and for drive-by contributions, git messages are likely to be highly ad hoc and idiosyncratic.” Another concern was the lack of time and tooling to support SECOM in daily workflows—highlighting the need for automation, which motivated the design of RQ3.

B. LLMs Impact in Completeness and Standardization (RQ3)

Writing commit messages and adhering to a standard can be time-consuming for practitioners. In response to concerns raised during our feedback sessions—and motivated by the broader challenge of ensuring completeness and standardization in security commit messages—we investigated the potential of large language models (LLMs), specifically “LLaMa 3.2”, to assist in generating key components of commit messages. **Methodology.** We selected a random sample of 2,207 commit messages—representing 20% of the dataset used to answer RQ1 and RQ2—referred to as the Original (OR) set. From this sample, we constructed two new subsets of commit messages: one using a mixed approach that combines metadata fields collected from vulnerability reports (i.e., real data) with LLM-generated content, and another generated entirely by an LLM (including metadata fields). For both samples, we asked LLaMa to use the diff provided by the security patch and generate a commit message following the short form (Listing 2) with and without generating metadata fields. The prompts are available in our replication package at `4-llm-analysis/generate-datasets.ipynb`.

Datasets. Mixed Approach (MX): While LLMs are widely recognized for their effectiveness in code summarization, their ability to infer security metadata remains underexplored. In this approach, the header and body of the commit message were generated by the LLM based on the corresponding patch diffs, while metadata fields were sourced directly from the original dataset. The format used in the prompt to generate the MX messages did not include the metadata fields. Metadata fields were added from our original dataset and after generation Full LLM generated (FLLM): Header, vulnerability ID, body and metadata were all generated by the LLM.

Extraction. We used the original commit and metadata set to generate 2,207 messages for both approaches. Rather than evaluating LLM accuracy in inferring data or metadata, our goal was to assess whether combining available metadata with LLM-generated content leads to more complete commit messages. To do so, we applied the same extraction analysis methodology from RQ1 across all three datasets.

Listing 2: SECOM short form used in RQ3

```
vuln-fix: <header/subject> (<Vuln-ID>)

<body>
# What:    Describe the vulnerability.
# Why:    Explain its impact or risk.
# How:    Describe how it was fixed.
</body>

[For each identified weakness:]
Weakness:  <Weakness Name or CWE-ID>
Severity:  <Low | Medium | High | Critical>
CVSS:      <CVSS Score (0-10)>
[End Weakness Block]
```

Findings. Table VII presents extraction results across all datasets, focusing on six key categories: SEWORD, ACTION, FLAW, VULNID, SEVERITY, and CWEID. Other categories

TABLE VII: Extraction Results for RQ3 (OR-original, MX-mixed_approach, FLLM-full_llm_generated)

Category	OR-#ENT	OR-#COM	OR-%COM	MX-#ENT	MX-#COM	MX-%COM	FLLM-#ENT	FLLM-#COM	FLLM-%COM
SECWORD	3325	1340	26.8%	13756	2107	42.1%	11292	2057	41.1%
ACTION	2399	1404	28.1%	6388	1984	39.7%	2437	1104	22.1%
FLAW	946	564	11.3%	3890	1831	36.6%	5111	1819	36.4%
VULNID	331	252	5.0%	2212	2187	43.7%	831	816	16.3%
SEVERITY	20	18	0.4%	1850	1807	36.1%	1334	1288	25.8%
CWEID	4	4	0.1%	1929	1770	35.4%	1667	1378	27.6%
Total	7025	2061	93.4%	30025	2207	100%	22672	2201	99.7%

were excluded from this analysis due to their lower priority in short-form messages. Using LLMs to generate the header and body fields in the **mixed dataset (MX)** led to notable spikes in information, when compared with **OR**: SECWORD ($\uparrow$ from 3,325 to 13,756), FLAW ($\uparrow$ from 946 to 3,980), VULNID ($\uparrow$ from 331 to 2,212), SEVERITY ($\uparrow$ from 20 to 1,850) and CWEID ($\uparrow$ from 4 to 1,929). MX achieves 100% commit coverage, meaning every message contains at least one identifiable entity, improving from 93.4% in the original data. The fully **LLM-generated dataset (FLLM)** extracted 22,672 entities—more than triple the original dataset (OR), which had 7,025. It showed spikes in SECWORD ($\uparrow$ from 3,325 to 11,292) and FLAW ($\uparrow$ from 947 to 5,111), but significantly underperformed the mixed approach (MX) across all categories: VULNID ($\downarrow$ from 2,212 to 831), and SEVERITY ($\downarrow$ from 1,850 to 1,334).

Conclusions. LLMs can significantly enhance the richness and completeness of commit messages when paired with reliable metadata. While LLMs outperform original messages in generating descriptive content, they are less reliable when producing metadata from scratch—often hallucinating vulnerability IDs, CVSS scores, or other verifiable values. These findings support a hybrid approach: combining LLM-generated text with trusted metadata offers a practical, scalable solution for producing high-quality security commit messages while minimizing the risk of critical errors.

V. IMPLICATIONS AND CONSIDERATIONS

This section covers implications of our findings, ethical considerations, and guidance for SECOM application.

Patch Transparency is a double-edged sword: it builds trust among consumers [17] but can expose potential vulnerabilities [14]. Security concerns are often used to justify opacity, yet non-transparent processes frequently lead to misuse and hinder timely patch management. The CERT Coordinated Vulnerability Disclosure (CVD) guide recommends avoiding silent patches, as they reduce public awareness of fixes, erode developer trust, and undermine automated triage tools that cannot reason about absent information. Adequate documentation mitigates these risks. Detailed commit messages, vulnerability IDs, issue references, and specific security fixes enable both automated tools and security teams to assess and apply patches quickly—narrowing the window attackers have to exploit known vulnerabilities. The SECOM convention

proposed in this paper helps vendors produce better documentation, ultimately strengthening the patch management phase. While excessive transparency can carry risk, we argue that structured documentation improves the effectiveness and speed of automated patch management. With improved transparency, OSS users and organizations can access fixes up to a week before formal disclosures [41].

Balancing Transparency and Security Documentation. Experienced security engineers should determine when and how to disclose details. In private development systems, detailed security commit messages carry minimal risk (assuming no internal threats). However, when patching critical vulnerabilities in widely used open-source software, engineers must carefully calibrate the level of detail—providing enough for users to assess criticality, without enabling exploitation. We do not advocate including exploit details in documentation, as this directly aids attackers. Such information should be withheld until users have had sufficient time to deploy the patch.

The Burden of a New Standard. As noted by a participant in our feedback sessions, adopting new best practices is often perceived as an additional burden, but it is essential for addressing current challenges and preventing future risks. Well-structured documentation builds trust between stakeholders and enables faster patch management, reducing the exposure window for users. To minimize the overhead introduced by our proposed standard, our team is developing automated solutions for compliance validation and commit message generation—RQ2 and RQ3 represent our first efforts in these directions. SECOM can also guide other tools in generating consistent, high-quality commit messages [44], and security researchers are already adopting it in practice [45]: more than 2,000 commit messages produced in the study follow our standard.

Advisories and Potential Inconsistencies. SECOM is designed to complement, not replace, vulnerability advisories. Advisories provide high-level vulnerability information but lack detail about specific security changes in a patch. Linking only to an advisory in a commit message does not give developers or automated tools sufficient context to understand the fix’s scope and importance. Moreover, not all vulnerabilities have advisories, and advisories are sometimes published after a patch is released, creating a gap in documentation. Inconsistencies between advisories and commit messages may arise. To reduce redundancy and maintain consistency, SECOM links to official advisories via the vulnerability ID.

When an advisory is available, developers should avoid duplicating its content in the commit message. The automated solutions discussed earlier should review messages against updated advisories to ensure consistency (e.g., revised severity scores or expanded details).

VI. THREATS TO VALIDITY

We now discuss the study’s potential threats to validity.

External Validity. The commit messages analyzed in this study are derived primarily from open-source repositories and publicly disclosed vulnerabilities. Practices observed in the open-source community may not fully reflect those in private or commercial environments, where confidentiality and internal processes may lead to different documentation practices. We used feedback from the open-source security community to complement RQ1 and RQ2 in the design of SECOM. Although our participant sample is small, all respondents were experienced security engineers from the OpenSSF, ensuring the reliability of their input. Our analysis is limited to English-language commit messages; findings may not generalize to other languages. Additionally, we did not examine all rules specified in the referenced standards, so our conclusions may not capture the full scope of established best practices.

Internal Validity. Our extraction pipeline targets a predefined set of entity categories, and other relevant aspects of commit messages may have been overlooked. The rule-based NER approach, while efficient and transparent, may miss entities expressed in unexpected forms or novel terminology not covered by our patterns. Furthermore, the LLM evaluation (RQ3) used a single model (LLaMa 3.2) and a fixed prompting strategy; results may vary with different models or prompt designs. We caution that our findings may not be fully generalizable to all security commit message practices, but they provide grounded insights into key aspects that affect documentation quality and patch management.

VII. RELATED WORK

Software Security Patch Management. Security patches are considered the most effective strategy to mitigate software vulnerabilities [6], [10], [46]. Yet, the *lack of efficient patch triage systems* to identify and prioritize security patches remains a key challenge [3], [7], [11], [12]: current processes are largely manual and prone to overlooking important fixes. Silent fixes—vulnerabilities patched without any release note—exacerbate this problem by leaving users unaware of patches and their criticality [17]. Previous work has shown that 56.7% of security commit messages have medium or lower levels of informativeness, impeding triage effectiveness [18]. This paper proposes a solution to make security commit messages more informative, improving the identification and prioritization of security patches and reducing the time users are exposed to vulnerabilities.

Software Security Patch Detection. Over the past years, many approaches have emerged to detect security patches through metadata (e.g., commit logs, commit metadata, and associated reports) and code changes. Here, we only mention

approaches considering commit logs (or messages). *Reis et al.* detected security patches for 16 types of weaknesses in commit messages by applying different regular expressions per each kind of weakness [13]. *Zhou and Asankhaya* leveraged regular expressions to identify features for predicting security-relevant commits in commit logs, commit metadata, and associated bug reports. Features are vectorized and learned by word2vec [19]. *Sabetta and Bezzi* extended previous work by considering code changes as well [16]—which led to considerable performance improvements. *Sawadogo et al.* proposed SSPCatcher, a co-Training-based approach to catch security patches based on commit logs and code features as part of an automatic monitoring service of code repositories [12]. In this study, we do not propose a new model or approach to detect security patches. Instead, we empirically analyze security commit messages and propose a standard that can improve the effectiveness of the different triage systems tasks.

Security Commit Message Analysis. Previous work has shown that only 38% of commit messages used to “silently” patch software vulnerabilities included security-related words [14]. A study on 2k commit message shown that 23% of those commit messages were poorly document and not clearly related to security issues [47]. This study also listed some of the guidelines that could be used to create security commit messages. In the present work, we analyzed a larger set of commit messages (11k) than in [47] and do not focus only on silent fixes [14]. We also assess how best practices are applied to security commit messages and how we could use LLMs to make security commit messages more complete.

VIII. CONCLUSIONS AND FUTURE WORK

Our analysis of over 11k security commit messages reveals that critical metadata—including vulnerability identifiers, weakness classifications, severity levels, and detection methods—is frequently absent, and adherence to established best practices remains inconsistent. To address this, we propose SECOM, the first convention specifically designed for security commit messages, and demonstrate that combining LLM-generated content with trusted metadata significantly enhances message completeness and structure. In the future, we plan to release automated solutions for compliance validation and message generation to reduce the overhead SECOM introduces in the security patch lifecycle.

ACKNOWLEDGMENTS

We sincerely thank Hakan Erdogmus for his valuable feedback and insightful contributions during the early stages of this project. His involvement helped shape key ideas and provided a strong foundation for our work. We thank the Open-Source Security Foundation (OpenSSF) and the Open-Source Vulnerability Database teams for supporting and validating SECOM. This work was financed by National Funds through the Portuguese funding agency, FCT - Fundação para a Ciência e a Tecnologia (refs. UIDP/50021/2020 and CMU/TIC/0064/2019) and a PhD scholarship (ref. SFRH/BD/143319/2019).

REFERENCES

- [1] B. Thomas, “New windows vulnerabilities highlight patch management challenges,” <https://www.bitsight.com/blog/new-windows-vulnerabilities-highlight-patch-management-challenges>, January 2020.
- [2] R. Brandom, “Former equifax ceo blames breach on a single person who failed to deploy patch,” <https://www.theverge.com/2017/10/3/16410806/equifax-ceo-blame-breach-patch-congress-testimony>, October 2017.
- [3] T. H. News, “The continuing threat of unpatched security vulnerabilities,” <https://thehackernews.com/2022/03/the-continuing-threat-of-unpatched.html>, March 2022.
- [4] Cybersecurity and I. S. Agency, “Alert (aa22-047a) russian state-sponsored cyber actors target cleared defense contractor networks to obtain sensitive u.s. defense information and technology,” <https://www.cisa.gov/uscert/ncas/alerts/aa22-047a>, February 2022.
- [5] H. N. Security, “Majority of 2019 breaches were the result of unapplied security patches,” <https://www.helpnetsecurity.com/2019/10/30/unapplied-security-patches/>, October 2019.
- [6] M. Souppaya and K. Scarfone, “Guide to enterprise patch management technologies,” <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-40r3.pdf>, July 2013.
- [7] F. Li, L. Rogers, A. Mathur, N. Malkin, and M. Chetty, “Keepers of the machines: Examining how system administrators manage software updates for multiple machines,” in *SOUPS @ USENIX Security Symposium*. USENIX Association, 2019.
- [8] C. Tiefenau, M. Häring, K. Krombolz, and E. Von Zezschwitz, “Security, availability, and multiple information sources: Exploring update behavior of system administrators,” in *Proceedings of the Sixteenth USENIX Conference on Usable Privacy and Security*, ser. SOUPS’20. USA: USENIX Association, 2020.
- [9] “Survey on international standards and best practices for patch management of complex industrial control systems: The critical infrastructure of particle accelerators case study,” *Int. J. Crit. Comput.-Based Syst.*, vol. 9, no. 1–2, p. 115–132, jan 2019.
- [10] N. Dissanayake, A. Jayatilaka, M. Zahedi, and M. A. Babar, “Software security patch management - a systematic literature review of challenges, approaches, tools and practices,” *Information and Software Technology*, vol. 144, p. 106771, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950584921002147>
- [11] Z. Zhang, “An investigation of the android kernel patch ecosystem,” in *USENIX Security Symposium*, 2021.
- [12] A. D. Sawadogo, T. Bissyandé, N. Moha, K. Allix, J. Klein, L. Li, and Y. Le Traon, “Sspatcher: Learning to catch security patches,” *Empirical Software Engineering*, vol. 27, no. 6, p. 151, 2022. [Online]. Available: <https://doi.org/10.1007/s10664-022-10168-9>
- [13] S. Reis and R. Abreu, “Secbench: A database of real security vulnerabilities,” in *International Workshop on Secure Software Engineering in DevOps and Agile Development @ ESORICS*, 2017, pp. 69–85.
- [14] J. Zhou, M. Pacheco, Z. Wan, X. Xia, D. Lo, Y. Wang, and A. E. Hassan, “Finding a needle in a haystack: Automated mining of silent vulnerability fixes,” in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2021, pp. 705–716.
- [15] S. E. Ponta, H. Plate, and A. Sabetta, “Beyond metadata: Code-centric and usage-based analysis of known vulnerabilities in open-source software,” *CoRR*, vol. abs/1806.05893, 2018. [Online]. Available: <http://arxiv.org/abs/1806.05893>
- [16] A. Sabetta and M. Bezzi, “A practical approach to the automatic classification of security-relevant commits,” *CoRR*, vol. abs/1807.02458, 2018. [Online]. Available: <http://arxiv.org/abs/1807.02458>
- [17] A. Householder, G. Wassermann, A. Manion, and C. King, “CERT@ Guide to Coordinated Vulnerability Disclosure,” 9 2020. [Online]. Available: https://kilthub.cmu.edu/articles/report/CERT_Guide_to_Coordinated_Vulnerability_Disclosure/12367340
- [18] S. Reis, R. Abreu, and C. Pasareanu, “Are security commit messages informative? not enough!” in *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 196–199. [Online]. Available: <https://doi.org/10.1145/3593434.3593481>
- [19] Y. Zhou and A. Sharma, “Automated identification of security issues from commit messages and bug reports,” in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, ser. ESEC/FSE 2017. New York, NY, USA: Association for Computing Machinery, 2017, p. 914–919. [Online]. Available: <https://doi.org/10.1145/3106237.3117771>
- [20] Y. Tian, Y. Zhang, K.-J. Stol, L. Jiang, and H. Liu, “What makes a good commit message?” in *ICSE’22*. ACM, may 2022. [Online]. Available: <https://doi.org/10.1145/2F3510003.3510205>
- [21] “Conventional commits,” <https://www.conventionalcommits.org/en/v1.0.0/>, accessed April 9, 2026.
- [22] L. Torvalds, “Linus torvalds describes a good commit message,” <https://github.com/torvalds/subsurface-for-dirk/blob/a48494d2fbcd58c751e9b7e8fbff88582f9b2d02/README#L88>, accessed April 9, 2026.
- [23] S. Patterson, “Developer tip: Keep your commits “atomic”,” <https://www.freshconsulting.com/insights/blog/atomic-commits/>, accessed April 9, 2026.
- [24] C. Beams, “How to write a git commit message,” <https://cbea.ms/git-commit/>, accessed April 9, 2026.
- [25] W. Sun, Y. Miao, Y. Li, H. Zhang, C. Fang, Y. Liu, G. Deng, Y. Liu, and Z. Chen, “Source code summarization in the era of large language models,” in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 419–431.
- [26] R. Haldar and J. Hockenmaier, “Analyzing the performance of large language models on code summarization,” in *Proceedings of the 13th International Conference on Language Resources and Evaluation (LREC 2024)*. European Language Resources Association (ELRA), 2024, pp. 1234–1243.
- [27] J. Zhu, Y. Miao, T. Xu, J. Zhu, and X. Sun, “On the effectiveness of large language models in statement-level code summarization,” in *Proceedings of the 2024 IEEE 24th International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 2024, pp. 216–227.
- [28] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, “Few-shot training llms for project-specific code summarization,” in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2022, pp. 123–134.
- [29] NIST, “National vulnerability database,” <https://nvd.nist.gov/>, accessed April 9, 2026.
- [30] Google, “A distributed vulnerability database for open source,” <https://osv.dev/>, accessed April 9, 2026.
- [31] O. Chang and R. Cox, “Open source vulnerability format,” <https://ossf.github.io/osv-schema/>, September 2022.
- [32] A. Anwar, A. Abusnaina, S. Chen, F. Li, and D. Mohaisen, “Cleaning the nvd: Comprehensive quality assessment, improvements, and analyses,” *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 6, pp. 4255–4269, 2022.
- [33] C. Hughes, “(2024) Death knell of the nvd? Accessed: 2025-04-27. [Online]. Available: <https://www.resilientcyber.io/p/death-knell-of-the-nvd>
- [34] VulnCheck, “(2024) Danger is still lurking in the nvd backlog. Accessed: 2025-04-27. [Online]. Available: <https://vulncheck.com/blog/nvd-backlog-exploitation-lurking>
- [35] J. Li, A. Sun, J. Han, and C. Li, “A survey on deep learning for named entity recognition,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 1, pp. 50–70, 2022.
- [36] A. Mikheev, M. Moens, and C. Grover, “Named entity recognition without gazetteers,” in *Ninth Conference of the European Chapter of the Association for Computational Linguistics*. Bergen, Norway: Association for Computational Linguistics, Jun. 1999, pp. 1–8. [Online]. Available: <https://aclanthology.org/E99-1001>
- [37] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer, “Neural architectures for named entity recognition,” in *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. San Diego, California: Association for Computational Linguistics, Jun. 2016, pp. 260–270. [Online]. Available: <https://aclanthology.org/N16-1030>
- [38] K. Hakala and S. Pyysalo, “Biomedical named entity recognition with multilingual BERT,” in *Proceedings of the 5th Workshop on BioNLP Open Shared Tasks*. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 56–61. [Online]. Available: <https://aclanthology.org/D19-5709>
- [39] A. Gonzalez-Agirre, M. Marimon, A. Intxaurreondo, O. Rabal, M. Villegas, and M. Krallinger, “PharmaCoNER: Pharmacological substances, compounds and proteins named entity recognition track,” in *Proceedings of the 5th Workshop on BioNLP Open Shared Tasks*. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 1–10. [Online]. Available: <https://aclanthology.org/D19-5701>

- [40] Y. Dong, W. Guo, Y. Chen, X. Xing, Y. Zhang, and G. Wang, "Towards the detection of inconsistencies in public security vulnerability reports," in *Proceedings of the 28th USENIX Conference on Security Symposium*, ser. SEC'19. USA: USENIX Association, 2019, p. 869–885.
- [41] F. Li and V. Paxson, "A large-scale empirical study of security patches," in *CCS'17*. New York, NY, USA: Association for Computing Machinery, 2017, p. 2201–2215. [Online]. Available: <https://doi.org/10.1145/3133956.3134072>
- [42] R. Croft, D. Newlands, Z. Chen, and M. A. Babar, "An empirical study of rule-based and learning-based approaches for static application security testing," in *ESEM'21*, 2021. [Online]. Available: <https://doi.org/10.1145/3475716.3475781>
- [43] J. A. et al., "The open source software security mobilization plan," https://8112310.fs1.hubspotusercontent-na1.net/hubfs/8112310/OpenSSF/OSS%20Mobilization%20Plan.pdf?utm_referrer=https%3A%2F%2Fopenssf.org%2Foss-security-mobilization-plan%2F, 2022.
- [44] M. Linares-Vásquez, L. F. Cortés-Coy, J. Aponte, and D. Poshyvanyk, "Changescribe: A tool for automatically generating commit messages," in *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, vol. 2, 2015, pp. 709–712.
- [45] J. Leitschuh and P. Way, "Scaling the security researcher to eliminate oss vulnerabilities once and for all," <https://i.blackhat.com/USA-22/Thursday/US-22-Leitschuh-Scaling-the-Security-Researcher.pdf>, accessed April 9, 2026.
- [46] F. Araujo, K. W. Hamlen, S. Biedermann, and S. Katzenbeisser, "From patches to honey-patches: Lightweight attacker misdirection, deception, and disinformation," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 942–953. [Online]. Available: <https://doi.org/10.1145/2660267.2660329>
- [47] S. Reis, R. Abreu, H. Erdogmus, and C. Păsăreanu, "Secom: Towards a convention for security commit messages," in *MSR'22*, 2022, pp. 764–765.